

2. After setting these values, let's restart the primary PostgreSQL server on the physical server we have to execute:

```
# systemctl restart postgresql
```

With the container provided with the book, we can simply exit from the container with *Ctrl + D* and then restart it.

3. Once that is done, we will run this command from the shell:

```
# netstat -an | grep 5432
tcp 0 0 0.0.0.0:5432 0.0.0.0:* LISTEN
tcp6 0 0 :::5432 :::* LISTEN
unix 2 [ ACC ] STREAM LISTENING 19910 /var/run/
postgresql/.s.PGSQL.5432
```

As we can see, PostgreSQL now listens to all the network interfaces available on the server.

Replica server – postgresql.conf

When it comes to the replica server, the changes to `postgresql.conf` are as follows:

```
# Add settings for extensions here
max_logical_replication_workers = 4
max_worker_processes = 10
```

As we can see, the values of `listen_addresses` and `wal_level` are identical to the primary; here, we don't have the values for `max_replication_slots` and `max_wal_senders`, but we have the values for the following:

- `max_logical_replication_workers`: This parameter must be set to one per subscription, plus some values to consider for table synchronizations.
- `max_worker_processes`: This must be set to at least one for each replication worker, plus one.

Here, as we did with the primary, let's restart the PostgreSQL server:

```
# systemctl restart postgresql
```

Once restarted, run this command from the shell:

```
# netstat -an | grep 5432
tcp 0 0 0.0.0.0:5432 0.0.0.0:* LISTEN
tcp6 0 0 :::5432 :::* LISTEN
```

```
unix 2 [ ACC ] STREAM LISTENING 19910 /var/run/postgresql/.s.PGSQL.5432
```

As we can see, PostgreSQL now listens to all the network interfaces available on the server.

The `pg_hba.conf` file

Let's now configure this file on the primary server so that it is possible to connect the replica machine and the primary machine. On the primary machine, we set the following:

```
# IPv4 local connections:
host all all 127.0.0.1/32 md5
host all replicarole 192.168.144.2/32 md5
```

This allows the user to replicate them on the replica machine to query the primary server. To activate the change, it is necessary to reload the primary server:

```
# systemctl reload postgresql
```

Logical replication setup

At this point, we have everything ready to begin preparing our logical replica:

1. Let's go to the primary machine and create our database:

```
postgres=# create database db_source;
CREATE DATABASE
postgres=# \c db_source
You are now connected to database "db_source" as user "postgres"
```

2. Let's now create a table, `t1`, making sure that it has the primary key:

```
db_source=# create table t1 (id integer not null primary key, name
varchar(64));
CREATE TABLE
```

3. Let's give the `REPLICAROLE` user `SELECT` permissions:

```
db_source=# GRANT SELECT ON ALL TABLES IN SCHEMA public TO
replicarole;
GRANT
```

4. Now, let's create the publication on the primary machine, where we will indicate the list of tables that we want to replicate on the replica machine.

We can also indicate all the tables, as shown in our example:

```
db_source=# CREATE PUBLICATION all_tables_pub FOR ALL TABLES;
CREATE PUBLICATION
```

5. At this point, we go to the replica machine and create a new database:

```
postgres=# create database db_destination;
CREATE DATABASE
postgres=# \c db_destination
You are now connected to database "db_destination" as user
"postgres"
```

6. We recreate the exact structure of the table that we created in the primary machine:

```
db_destination=# create table t1 (id integer not null primary key,
name varchar(64));
CREATE TABLE
```

7. After this, we have to set the subscription so that the data from the publication is replicated on the replica machine:

```
db_destination=# CREATE SUBSCRIPTION sub_all_tables CONNECTION
'user=replicarole password=LearnPostgreSQL host=pg_pub port=5432
dbname=db_source' PUBLICATION all_tables_pub;
NOTICE: created replication slot "sub_all_tables" on publisher
CREATE SUBSCRIPTION
```

Now our logical replication setup is complete.

8. We can try to insert some data into the primary server:

```
db_source=# insert into t1 values(1,'Linux'),(2,'FreeBSD');
INSERT 0 2
```

9. As we can see here, the same data has been replicated on the replica server:

```
db_destination=# select * from t1;
 id |  name
----+-----
  1 | Linux
  2 | FreeBSD
(2 rows)
```

Thus, we have successfully prepared our logical replica. We will now learn how to monitor it in the next section.

Monitoring logical replication

Just as it does for physical replication, PostgreSQL provides the necessary tools to monitor logical replication.

For logical replication, we must query the `pg_stat_replication` table, which is the same table used to monitor physical replication, as we can see here:

```
db_source=# \x
Expanded display is on.
db_source=# select * from pg_stat_replication ;
-[ RECORD 1 ]-----+-----
pid           | 144
usesysid      | 16477
username      | replicarole
application_name | sub_all_tables
client_addr   | 192.168.144.2
client_hostname |
client_port   | 43162
backend_start  | 2023-06-16 15:04:09.074749+00
backend_xmin   |
state         | streaming
sent_lsn      | 0/1DD0398
write_lsn     | 0/1DD0398
flush_lsn     | 0/1DD0398
replay_lsn    | 0/1DD0398
write_lag     |
flush_lag     |
replay_lag    |
sync_priority  | 0
sync_state    | async
reply_time    | 2023-06-16 15:05:23.524003+00
```

The information shown by this query is the same as what we saw in the case of physical replication, but we know this information refers to a logical replica because we have the slot name `sub_all_tables`, which we created before, on the `application_name`.

This query must be performed on the primary server (pg_pub). If we run the same query on the replica machine (pg_sub), we do not get any results, as we can see here:

```
db_destination=# select * from pg_stat_replication ;
(0 rows)
```

There are also two other catalog tables that we can query for more information about publications and subscriptions. Let's say that, on the primary server, we perform this:

```
db_source=# select * from pg_publication;
-[ RECORD 1 ]+-----
oid          | 16479
pubname      | all_tables_pub
pubowner     | 10
puballtables | t
pubinsert    | t
pubupdate    | t
pubdelete    | t
pubtruncate  | t
pubviaroot   | f
```

If we do that, we get information about all publications created in the database. For more information about this, consult the official documentation: <https://www.postgresql.org/docs/current/catalog-pg-publication.html>.

Similarly, let's say we run this query on the replica server:

```
db_destination=# select * from pg_subscription;
-[ RECORD 1 ]-----+-----
oid          | 16477
subdbid      | 16471
subskiplsn   | 0/0
subname      | sub_all_tables
subowner     | 10
subenabled   | t
subbinary    | f
substream    | f
subtwophasestate | d
subdisableonerr | f
```

```
subconninfo      | user=replicarole password=LearnPostgreSQL host=pg_pub
port=5432 dbname=db_source
subslotname      | sub_all_tables
subsynchronous   | off
subpublications  | {all_tables_pub}
```

We then have information about all subscriptions created in the database. For more information about this, consult the official documentation: <https://www.postgresql.org/docs/current/catalog-pg-subscription.html>.

Read-only versus write-allowed

In the previous chapter, we saw that we can access a physical replication server only using read operations and that write operations are not allowed. We also saw that physical replication replicates any type of operation, both DML operations and DDL operations. Using logical replication, we can also access write operations on the replica server, but in a logical replica, only DML operations are replicated to the replica server; the DDL operations are not replicated. Let's conduct some tests and see what happens. In the following examples, the primary server will always be called `pg_pub`, and the server with logical replication will always be called `pg_sub`.

This is our initial situation on the `pg_pub` server:

```
db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
(2 rows)
```

This is our initial situation on the `pg_sub` server:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
(2 rows)
```

Let's insert a record on the `pg_sub` server:

```
db_destination=# insert into t1 values (3,'OpenBSD');
INSERT 0 1
```

This is now the situation on the `pg_sub` server:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  3 | OpenBSD
(3 rows)
```

On the `pg_pub` server, we still have the following:

```
db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
(2 rows)
```



Note that the logical replica allows write operations on the replica server.

Let's see what happens if we add one record to the `pg_pub` server:

```
db_source=# insert into t1 values(4,'Minix');
INSERT 0 1
```

The situation on the `pg_pub` server is as follows:

```
db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
(3 rows)
```

The situation on the `pg_sub` server is as follows:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  3 | OpenBSD
  4 | Minix
(4 rows)
```

As we can see, the values have been inserted in the table of the primary server `pg_pub` and replicated through the logical replica on the `pg_sub` server. Let's now see what happens if we try to insert a record with a key value already inserted on the `pg_sub` server. For example, let's try to insert this record:

```
db_source=# insert into t1 values(3,'Windows');
INSERT 0 1
```

The situation on the `pg_pub` server is now this:

```
db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
  3 | Windows
(4 rows)
```

However, the situation on the `pg_sub` server is now this:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  3 | OpenBSD
  4 | Minix
(4 rows)
```

No record has been inserted on the `pg_sub` server. If we are not in a container environment, we can examine the `postgresql.log` file of the `pg_sub` replica server; otherwise, if we use the `Docker chapter18_logical_clear` environment, we can open another bash terminal window and execute the following two statements to see the log:

```
$ cd chapter18_logical_clear
$ chapter18_logical_clear$ sudo docker-compose logs -f

learn_postgresql_sub_1 | 2023-06-16 15:17:23.774 UTC [213] ERROR:
duplicate key value violates unique constraint "t1_pkey"
learn_postgresql_sub_1 | 2023-06-16 15:17:23.774 UTC [213] DETAIL: Key
(id)=(3) already exists.
learn_postgresql_sub_1 | 2023-06-16 15:17:23.774 UTC [213] CONTEXT:
processing remote data for replication origin "pg_16477" during message
type "INSERT" for replication target relation "public.t1" in transaction
780, finished at 0/1DD0918
learn_postgresql_sub_1 | 2023-06-16 15:17:23.776 UTC [1] LOG: background
worker "logical replication worker" (PID 213) exited with exit code 1
```

If we examine the log of the `pg_pub` primary server, we will see that there are the following messages:

```
learn_postgresql_pub_1 | 2023-06-16 15:17:23.774 UTC [221] LOG: logical
decoding found consistent point at 0/1DD0720
learn_postgresql_pub_1 | 2023-06-16 15:17:23.774 UTC [221] DETAIL: There
are no running transactions.
learn_postgresql_pub_1 | 2023-06-16 15:17:23.774 UTC [221] STATEMENT:
START_REPLICATION SLOT "sub_all_tables" LOGICAL 0/1DD0638 (proto_version
'3', publication_names '"all_tables_pub"')
```

The duplicate key error on the replica server has the effect of causing the message illustrated here on the primary server.

So now, if we try to add another record on the primary server, this record will not be inserted on the replica server. Let's say we tried on the `pg_pub` server to perform this statement:

```
db_source=# insert into t1 values(5,'Unix');
INSERT 0 1
```

We would then have this on the `pg_pub` server:

```
db_source=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  4 | Minix
  3 | Windows
  5 | Unix
(5 rows)
```

In the replica `pg_sub` server, though, we would still have this:

```
db_destination=# select * from t1;
 id | name
----+-----
  1 | Linux
  2 | FreeBSD
  3 | OpenBSD
  4 | Minix
(4 rows)
```

From now on, logical replication no longer replicates data, and if we execute this query on the `pg_pub` server:

```
db_source=# select * from pg_stat_replication;
(0 rows)
```

No more replication will be found; that's because our logical replication no longer works.



If we want to write records on the replica server, we have to make sure that these records do not conflict with the records on the primary server.

A simple way to realign our replica server is to drop the subscription, truncate the table, and make the subscription again:

```
db_destination=# drop subscription sub_all_tables ;
NOTICE:  dropped replication slot "sub_all_tables" on publisher
```